

Department of Statistics & Computer Science
University of Kelaniya
Academic Year -2024/2025 (Semester II)

COSC 32133/BECS 32263

Full-Stack Software Development

Assignment 01

Bank Transactions with @Transactional, Rollback and @Query

Build a **Bank Transfer System** in which money is moved between two customers. Use @Transactional so that a transfer is either completed fully or **rolled back** completely when anything fails and write your own database queries with the @Query annotation in the repository interfaces. The data is stored in **MySQL,** and the code is published to **GitHub**.

1. Learning Outcomes

By the end of this lab, you will be able to:

- Explain the ACID properties and why a bank transfer must run inside one transaction.
- Use @Transactional and explain its rollback rules, readOnly and propagation (REQUIRED, REQUIRES_NEW).
- Demonstrate a rollback by forcing a failure in the middle of a transfer.
- Write user-defined queries with @Query: JPQL, native SQL, aggregate and UPDATE queries (@Modifying).
- Use @Param to pass named parameters into a query.
- Store money correctly using Big Decimal.
- Record failed transfers in a separate transaction so that they are not lost in the rollback.
- Publish the project to GitHub using a feature branch and a Pull Request.

2. Scenario

A small bank wants a web page where a clerk can move money from one customer's account to another. A transfer has two parts: **withdraw** from the sender and **deposit** to the receiver. If the system fails after the withdrawal but before the deposit, the money will simply vanish. To prevent this, both parts must be one **transaction**: if any step fails, the database returns to the state it had before the transfer started.

The application shows the customers and their balances, a transfer form, the latest transactions (successful and failed), and the transaction history of each account. A special checkbox lets you **simulate a crash** so you can see a rollback happen.

3. Prerequisites

- **JDK 17 or later** (21 recommended) and an IDE with Lombok support.
- **MySQL** running on your machine: XAMPP (MySQL module) or MySQL Server.
- Git and a GitHub account.

4. Key Concepts

4.1 What is a transaction? (ACID)

Property	Meaning in our bank example
Atomicity	All steps happen or none do. Withdraw + deposit + saving the record is a single unit.
Consistency	The database always follows the rules. The total money in the bank does not change by a transfer.
Isolation	Two transfers running at the same time do not see each other's half-finished work.
Durability	Once a transfer is committed it is permanently saved, even if the server crashes afterwards.

4.2 How @Transactional works

When a method is annotated with @Transactional, Spring wraps the bean in a **proxy**. The proxy starts a database transaction before the method runs, **commits** it if the method finishes normally, and **rolls it back** if the method throws an exception.

```
Controller  --->  [ Spring proxy ]  --->  BankService.transfer()
                  |  BEGIN transaction
                  |     debit(sender)     <- UPDATE
                  |     credit(receiver)  <- UPDATE
                  |     save(record)      <- INSERT
                  |  no exception  ->  COMMIT   (all saved)
                  |  exception     ->  ROLLBACK (all undone)
```

Attribute	Meaning
@Transactional	Default: propagation REQUIRED (join the current transaction or start a new one). Rolls back on Runtime Exception and Error only.
rollbackFor = Exception.class	Also roll back for **checked** exceptions (e.g. IO Exception, SQL Exception).
readOnly = true	Tells Spring/database the method only reads data (safer and slightly faster).
propagation = REQUIRED	Use the caller's transaction if there is one; otherwise create one (default).
propagation = REQUIRES_NEW	Always suspend the caller's transaction and start a **new, independent** one that commits on its own.
propagation = SUPPORTS	Join a transaction if one exists; otherwise run **without** a transaction.

> **Common mistakes:** (1) Catching the exception **inside** the @Transactional method and not re-throwing it, so Spring never sees the failure and commits. (2) Calling a @Transactional method from another method **of the same class**: the proxy is bypassed, so no transaction starts. (3) Putting @Transactional on a private method; it is ignored. (4) Expecting a rollback for a checked exception without rollbackFor.

4.3 What is @Query?

Spring Data JPA can build simple queries from method names (findByEmail). For anything more complex you write the query yourself with @Query on a repository method. The query can be written in two languages:

Type	Example	Notes
JPQL (default)	SELECT c FROM Customer c WHERE c.balance > :min	Uses **class and field names** (Customer, balance). Portable across databases.
Native SQL	@Query(value = "SELECT COUNT(*) FROM customers", nativeQuery = true)	Uses **table and column names**. Real MySQL SQL.
UPDATE / DELETE	@Modifying @Query("UPDATE Customer c SET ...")	Needs @Modifying and runs inside a transaction. Returns the number of rows changed.

Parameters are passed by name: write :accountNumber in the query and @Param("accountNumber") on the method parameter.

5. Application Overview

Layer	File	Responsibility
View	bank.html, history.html	Dashboard (forms, tables, statistics) and account history page
Controller	BankController.java	Handles requests; on failure records a FAILED entry
Service	BankService.java	transfer() with @Transactional (the heart of this lab)
Service	TransactionAuditService.java	Saves FAILED records in a **new** transaction (REQUIRES_NEW)
Repository	CustomerRepository.java, BankTransactionRepository.java	All user-defined @Query queries
Entity	Customer, BankTransaction, TransactionStatus	Tables customers and bank_transactions
Other	TransferRequest, exceptions, DataLoader	Form object, custom errors, sample data

Steps inside one transfer (BankService.transfer):

Step	Action	Fails when
1	Validate: accounts different and both exist	Same account, or account not found
2	debit(sender, amount): UPDATE the sender's balance	Balance too low (0 rows updated)
3	(Experiment) throw a simulated failure	Checkbox ticked
4	credit(receiver, amount): UPDATE the receiver's balance	Database error
5	Save a SUCCESS record in bank_transactions	Database error

URLs:

Method	URL	Purpose
GET	/	Dashboard; optional ?keyword= searches customers by name
POST	/customers	Add a customer
POST	/transfer	Transfer money
GET	/accounts/{accountNumber}/history	Transactions of one account

6. Tasks

Task 1: Create the Project with Spring Initializer

1. Open **https://start.spring.io** and fill in the metadata:

Setting	Value
Project / Language	Maven / Java
Spring Boot	Latest stable release (not SNAPSHOT / M / RC)
Group	com.example
Artifact / Name	bank-transfer-app
Description	Bank Transfer System
Package name	com.example.bankapp
Packaging / Java	Jar / 21 (or 17)

2. Add these **six dependencies** (**ADD DEPENDENCIES** or Ctrl + B):

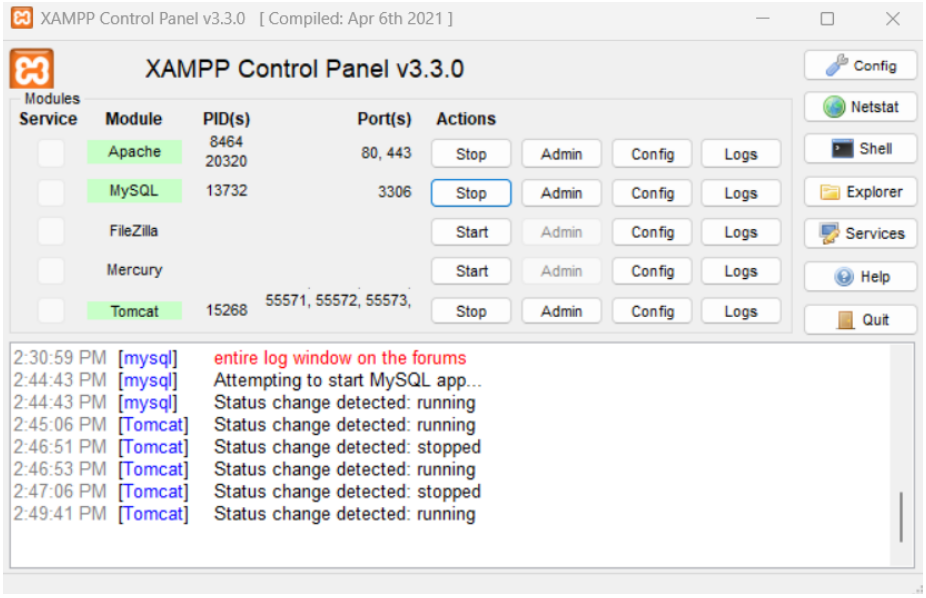
Dependency	Added to pom.xml	Why
Spring Web	spring-boot-starter-web (-webmvc in Boot 4)	Spring MVC and embedded Tomcat
Spring Data JPA	spring-boot-starter-data-jpa	Hibernate, repositories, @Query, @Transactional support
MySQL Driver	mysql-connector-j	Connects Java to MySQL

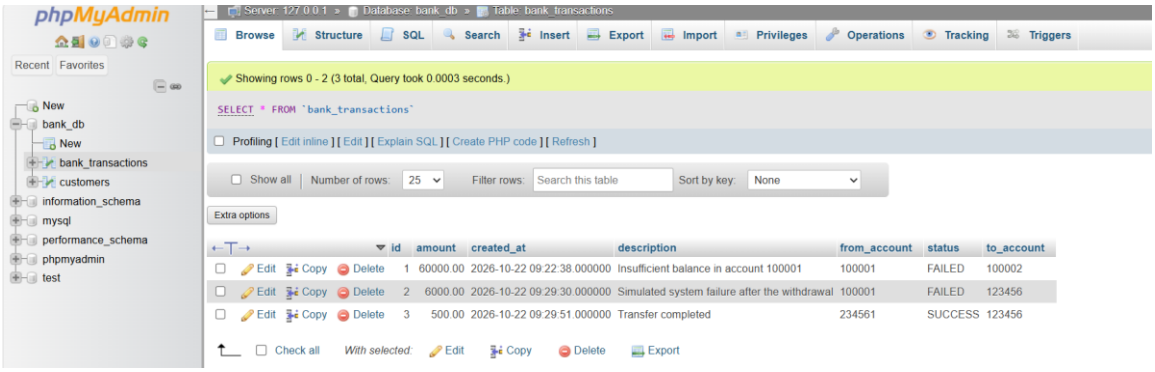
Dependency	Added to pom.xml	Why
Validation	spring-boot-starter-validation	@NotBlank, @DecimalMin, ...
Lombok	lombok	Generates getters, setters, constructors
Spring Boot DevTools	spring-boot-devtools	Automatic restart

3. Click **GENERATE**, extract the zip, and open the bank-transfer-app folder in your IDE. Enable Lombok annotation processing if your IDE needs it (see Lab Sheet 04).
4. Compare your pom.xml with **Appendix A**.

Task 2: Start MySQL, Configure the Application and Push to GitHub

1. Open the **XAMPP Control Panel** and click **Start** next to **MySQL** (or start your MySQL service).





2. Open src/main/resources/application.properties and replace its content. With XAMPP the user is root and the password is empty. The createDatabaseIfNotExist=true option creates the database bank_db automatically:

3. Create a GitHub repository named bank-transfer-app (no README, no .gitignore) and push the initial project:

4. Create your feature branch for the rest of the lab:

> **Tip:** The two logging.level lines make Spring print every transaction **begin, commit and rollback** in the console. You will use them in Task 9.

Task 3: Create Entities

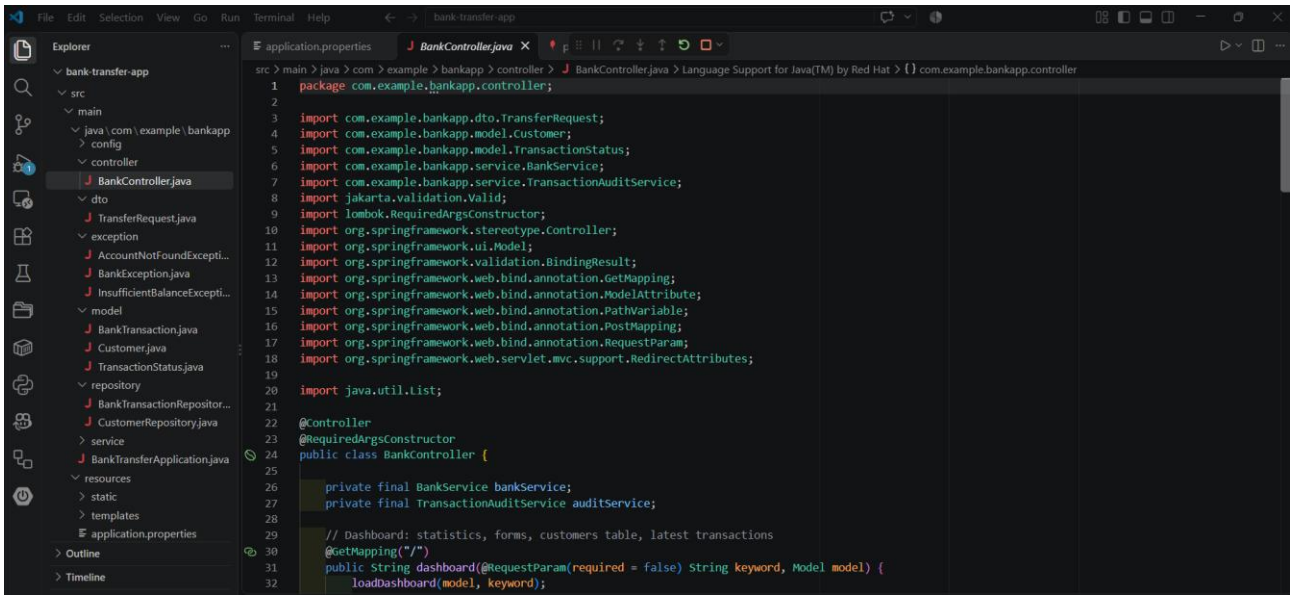
Create the packages model, repository, service, controller, dto, exception and config inside com.example.bankapp. In model create three files.

3.1 TransactionStatus.java (enum)

3.2 Customer.java

3.3 BankTransaction.java: one row for every transfer, successful or failed.

> **Why BigDecimal?** Never use double or float for money. They cannot represent values like 0.10 exactly, which causes rounding errors. BigDecimal is exact. @Column(precision = 14, scale = 2) creates a DECIMAL(14,2) column in MySQL.



Task 4: Create the Custom Exceptions

In the exception package create three small classes. They extend RuntimeException, so Spring **automatically rolls back** when they are thrown.

Task 5: Write the Repositories with @Query

This is where you write **user-defined queries**. Create CustomerRepository in the repository package. It contains eight queries of five different kinds:

#	Method	Kind	What it does
1	getByAccountNumber	JPQL + @Param	Finds one customer by account number
2	countAccounts, countEmails	JPQL COUNT	Checks whether an account / email already exists
3	findAllOrdered	JPQL ORDER BY	All customers sorted by name

#	Method	Kind	What it does
4	searchByName	JPQL LIKE	Case-insensitive name search
5	getTotalBalance	JPQL aggregate	Sum of all balances (null if no customers)
6	countCustomersNative	**Native SQL**	Plain MySQL: SELECT COUNT(*) FROM customers
7	debit	UPDATE + @Modifying	Subtracts money only if balance >= :amount; returns rows changed
8	credit	UPDATE + @Modifying	Adds money to an account

> **Why is debit written as an UPDATE with a condition?** "Check the balance, then subtract" done in two steps is unsafe: another transfer could change the balance in between. The single statement ... WHERE balance >= :amount checks and updates **atomically**. If it returns 0, the balance was too low. @Transactional on these methods means that they join the transfer's transaction, and they also work correctly (as separate mini-transactions) if no outer transaction exists. clearAutomatically refreshes Hibernate's cache after the bulk update.

Now create BankTransactionRepository:

#	Method	What it does
1	findAllLatest	All transactions, newest first
2	findHistory	Transactions where the account is the sender **or** the receiver (OR condition)
3	findByStatusLatest	Filters by status; an **enum** can be passed as a parameter
4	countByStatus	Number of SUCCESS or FAILED transactions for the dashboard

Task 6: Write the Services with @Transactional

6.1 BankService.java in the service package. Study the comments, especially on transfer ():

Annotation	Effect here
@Transactional(readOnly = true) on the class	Default for all methods: read-only transaction (reports, searches)
@Transactional(rollbackFor = Exception.class) on transfer	Overrides the class default with a read-write transaction that rolls back for **any** exception
@Transactional on createCustomer	Read-write transaction for saving a customer
@RequiredArgsConstructor (Lombok)	Generates the constructor that injects the two repositories

6.2 TransactionAuditService.java: saves a FAILED record. It must be a separate class, because calling a @Transactional method of the same class would bypass the proxy.

> **Why REQUIRES_NEW?** When transfer() fails, its transaction is rolled back, which would also erase a FAILED record saved in the same transaction. REQUIRES_NEW saves the FAILED record in its own transaction that commits independently, so the failure is still recorded for auditing.

Task 7: Create the Form Object and the Controller

7.1 TransferRequest.java in the dto package holds the data typed in the transfer form:

7.2 BankController.java in the controller package. Notice how transfer() calls the service inside try, and the catch block runs after the rollback:

Task 8: Sample Data

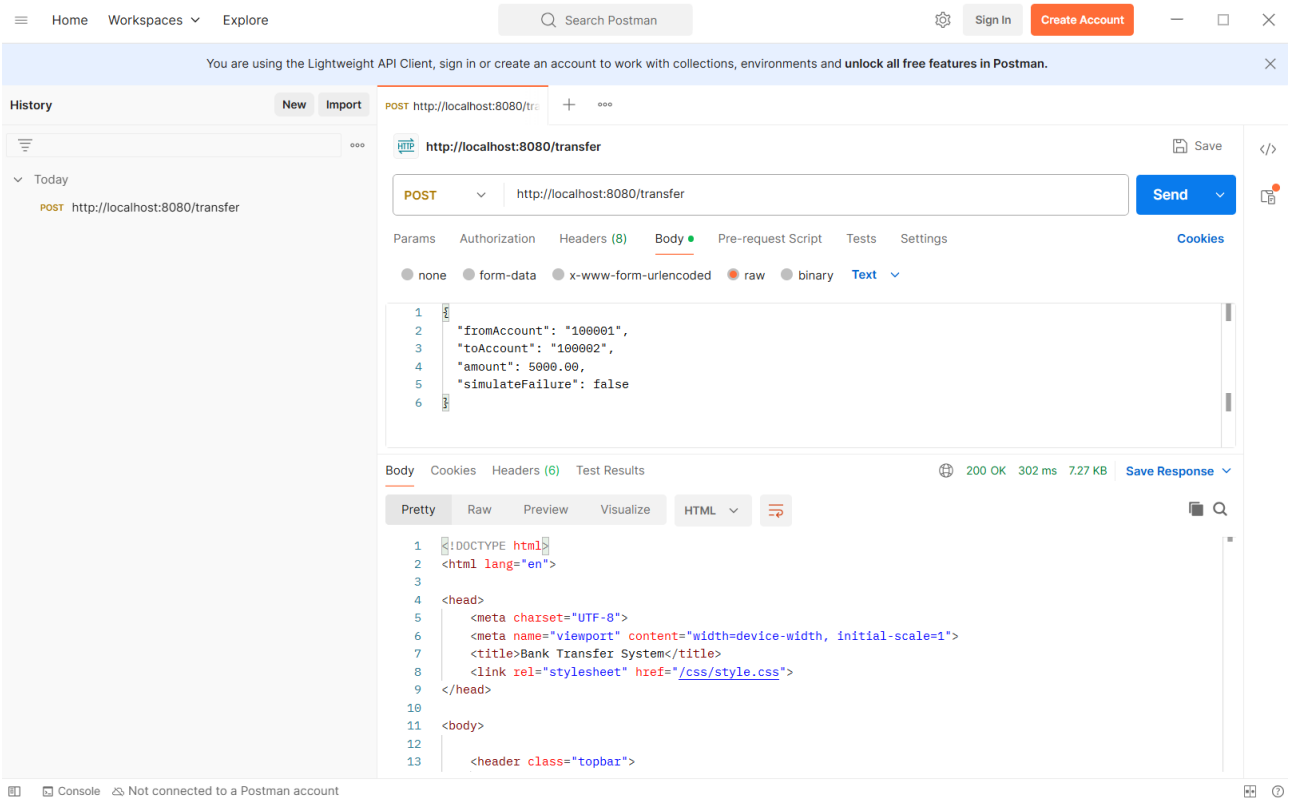
8.1 DataLoader.java in the config package inserts three customers the first time the application runs (total money = LKR 85,000.00)

Task 9: Run the Application and Test Commits and Rollbacks

1. Make sure MySQL is running, then start the application:

2. Run the tests below **in order using postman**. After each one, check the balances, the statistics and the **Latest Transactions** table, and read the console log (look for the words Initiating transaction commit or Initiating transaction rollback).

#	Action	Expected result	What it proves
1	Transfer **5,000** from Kamal (100001) to Nimali (100002).	Kamal 45,000.00; Nimali 30,000.00; total still 85,000.00; a SUCCESS row.	Commit: all steps saved
2	Transfer **20,000** from Saman (10,000 balance) to Kamal.	Red message; **no balance changes**; a FAILED row "Insufficient balance".	Exception → rollback
3	Choose the **same account** as sender and receiver.	Error "must be different accounts"; balances unchanged; FAILED row.	Validation inside the transaction
4	Transfer **1,000** from Kamal to Saman with **Simulate a system failure** ticked.	Error "All changes were rolled back". **Kamal is still 45,000.00** and Saman 10,000.00; a FAILED row.	The withdrawal that already happened was UNDONE
5	Submit with amount empty, then with amount 0.	Validation messages under the field; nothing saved.	Bean validation
6	Add a customer 100004 "Nuwan" with 15,000. Then try the same account number again.	Customer added; second attempt shows "already exists".	Custom @Query checks
7	Search "sil" in the customers box; click **View** for Kamal.	Only matching customers; history shows Kamal's SUCCESS and FAILED rows.	@Query LIKE and OR queries



4. **Verify in MySQL.** Open phpMyAdmin (XAMPP: **Admin** next to MySQL) or the MySQL terminal, select bank_db, and compare the data with the web page:

```
USE bank_db;
SELECT * FROM customers;
SELECT * FROM bank_transactions ORDER BY id DESC;
SELECT SUM(balance) FROM customers;
```

5. Check that after test 4 the sum of balances is unchanged and that bank_transactions contains FAILED rows even though those transfers were rolled back.

> **Look at the log:** For a failed transfer the console shows the transaction being created for BankService.transfer, then the exception, then **Initiating transaction rollback**. For the FAILED record you will also see a **second, separate transaction** (REQUIRES_NEW) being started and committed.

Task 10: Experiment: What Happens Without @Transactional?

Now remove the protection and see why transactions matter. Note the current **total balance** shown on the dashboard first.

1. In BankService, add the import org.springframework.transaction.annotation.Propagation and change the annotation of transfer to run **without** a transaction:

```
// @Transactional(rollbackFor = Exception.class)
@Transactional(propagation = Propagation.SUPPORTS)
public BankTransaction transfer( ... )
```

2. Restart the application. Transfer **1,000** from Kamal to Saman with **Simulate a system failure** ticked.

3. Observe: Kamal's balance is now **1,000 lower**, Saman received nothing, and the **total balance dropped by 1,000**. The money has vanished, because the withdrawal was committed on its own and nothing undid it. Take a screenshot.
4. Put the original annotation back (@Transactional(rollbackFor = Exception.class)), restart, and repeat the same transfer. The balances must stay unchanged this time.
5. Repair the lost money in MySQL so that your data is correct again:

```
UPDATE customers SET balance = balance + 1000 WHERE account_number = '100001';
```

Task 11: Push to GitHub, Pull Request and Merge

1. Push your feature branch:
2. On GitHub click **Compare & pull request** (base: main, compare: feature/bank-transfer). Title: 'Add bank transfer with @Transactional and @Query'. Describe the repository queries and the rollback behaviour, then click **Create pull request**.
3. In **Files changed** confirm that all Java files, templates and CSS are included and that target/ and secrets.properties are **not** included.
4. Click **Merge pull request → Confirm merge**.
5. Update your local main:

Task 12 : Extend the Application

- Add a @Query in BankTransactionRepository that returns transfers **above a given amount** and show them on a new page /large-transfers?min=10000.
- Add a @Query that returns the **total amount sent** by an account (SUM of SUCCESS transfers where fromAccount = :account) and show it on the history page.
- Add **Deposit** and **Withdraw** forms that use credit() and debit(), each inside a @Transactional service method, and record them in bank_transactions.
- Add a transfer fee: 1% is withdrawn from the sender and added to a bank account 999999. Make sure that all three updates are one transaction.

7. Troubleshooting

Problem / error message	Solution
Communications link failure	MySQL is not running. Start it in XAMPP and check the port (3306) in the JDBC URL.
Access denied for user 'root'	Your MySQL root user has a password. Put it in secrets.properties (Task 2, step 3).
Rollback does not happen (balances change after a failure)	Check that transfer has @Transactional from org.springframework.transaction.annotation (**not** jakarta.transaction), that the exception is not caught inside transfer, and that the tables use InnoDB.
TransactionRequiredException: Executing an update/delete query	@Modifying queries need a transaction. Keep @Transactional on debit / credit.
Failed records are missing	recordFailure must be in a **separate class** (TransactionAuditService) with REQUIRES_NEW.
Red errors on getters / @RequiredArgsConstructor	Lombok is not active. Enable annotation processing and reload the project.

Problem / error message	Solution
Could not resolve parameter / Named parameter not bound	The name in :accountNumber must match @Param("accountNumber") exactly.
QueryCreationException at start-up	A JPQL query uses a wrong class or field name. Use the **Java** names (Customer, fullName) in JPQL, not table or column names.
Whitelabel error / template error	Template file names must be bank.html and history.html in src/main/resources/templates.
Port 8080 already in use	Stop the other program or set server.port=8081.

8. Reference

Annotations used in this lab

Annotation	Purpose
@Transactional	Run a method inside a database transaction (commit / rollback)
@Transactional(rollbackFor = ...)	Also roll back for the listed (checked) exceptions
@Transactional(readOnly = true)	Read-only transaction for queries
@Transactional(propagation = REQUIRES_NEW)	Start a new independent transaction
@Query	Define your own JPQL or native SQL query on a repository method
@Param	Bind a method parameter to a named query parameter
@Modifying	Marks an @Query that is an UPDATE / DELETE
@Getter @Setter @NoArgsConstructor @RequiredArgsConstructor	Lombok: generate boilerplate code
@Entity @Id @Column @Enumerated	JPA mapping of classes and fields to tables and columns
@Valid + @NotBlank @NotNull @DecimalMin @Digits	Bean validation of form input
CommandLineRunner	Code that runs once when the application starts (sample data)

JPQL compared with SQL

	JPQL	Native SQL
Refers to	Entity class and field names	Table and column names
Example	SELECT c FROM Customer c WHERE c.fullName = :n	SELECT * FROM customers WHERE full_name = :n
Portable between databases	Yes	No (database specific)
Needs nativeQuery = true	No	Yes

Submission Requirements

- Submit as a PDF including all the screenshots and code.
- Include your name, student ID, date, course code, course name and your **GitHub repository URL**.
- Answer in your report: (a) What would have happened to the money in test 4 without @Transactional? (b) Why is record Failure in a separate class with REQUIRES_NEW? (c) Why does debit use UPDATE ... WHERE balance >=: amount instead of reading the balance first?
- The filename should be named as 'Assignment01_{your student ID}.pdf'
- Deadline: 11th of November 2026